PART 4

Databases

25 Database Indexing
26 Query Optimization
27 N+1 Queries
28 Connection Pooling
29 Read Replicas
30 Sharding
31 Partitioning
32 Replication
33 Leader Election

#025

Database Indexing

WHAT IT IS

A data structure (usually a B-tree) that lets the DB find rows without scanning the whole table.

WHY IT MATTERS

The #1 fix for slow queries. Costs: extra storage and slower writes.

EXAMPLE

```
Schema::table('tasks', function (Blueprint $t) {
    $t->index(['project_id', 'status', 'due_at']);
    // matches WHERE project_id=? AND status=? ORDER
    BY due_at
});
```

#026

Query Optimization

WHAT IT IS

Making queries do less work: select fewer columns/rows, use indexes, avoid functions on indexed columns.

WHY IT MATTERS

Query time dominates most request time.

EXAMPLE

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT id, title FROM tasks WHERE project_id = 42
    AND status = 'open' ORDER BY due_at LIMIT 20;
```

#027

N+1 Queries

WHAT IT IS

One query to load a list, then one extra query per item for a relation.

WHY IT MATTERS

100 rows = 101 queries; it kills performance silently.

EXAMPLE

```
// Bad
foreach (Task::all() as $t) echo $t->assignee->name;

// Good
foreach (Task::with('assignee:id,name')->get() as
    $t) echo $t->assignee->name;

// Catch it in dev
Model::preventLazyLoading(! app()->isProduction());
```

#028

Connection Pooling

WHAT IT IS

Reusing a limited set of DB connections instead of opening one per request.

WHY IT MATTERS

PostgreSQL connections are heavy (~10MB each); hundreds of PHP workers can exhaust max_connections.

EXAMPLE

```
; pgbouncer.ini
pool_mode = transaction
default_pool_size = 20
max_client_conn = 1000
```

#029

Read Replicas

WHAT IT IS

Copies of the primary DB that serve read-only queries.

WHY IT MATTERS

Scales read-heavy workloads and offloads reporting.

EXAMPLE

```
// config/database.php
'pgsql' => [
    'read'  => ['host' => ['10.0.0.21',
    '10.0.0.22']],
    'write' => ['host' => ['10.0.0.20']],
    'sticky' => true, // reads right after a write
    use the primary
],
```

#030

Sharding

WHAT IT IS

Splitting data across multiple databases by a key (e.g. tenant_id).

WHY IT MATTERS

Scales writes and storage beyond one machine; adds major complexity (cross-shard queries, rebalancing).

EXAMPLE

```
function connectionFor(Tenant $t): string {
    return 'shard_' . ($t->id % 4);
}
DB::connection(connectionFor($tenant))->table(
    'tasks')->get();
```

#031

Partitioning

WHAT IT IS

Splitting one big table into smaller physical pieces inside the same database.

WHY IT MATTERS

Faster scans, easy deletion of old data (drop a partition), better maintenance.

EXAMPLE

```
CREATE TABLE events (id bigserial, created_at
    timestamptz NOT NULL, payload jsonb)
  PARTITION BY RANGE (created_at);
CREATE TABLE events_2026_10 PARTITION OF events
  FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
```

#032

Replication

WHAT IT IS

Continuously copying changes from a primary to one or more replicas.

WHY IT MATTERS

High availability, read scaling, backups, analytics.

EXAMPLE

```
-- on primary
CREATE PUBLICATION app_pub FOR TABLE tasks, projects;
-- on subscriber
CREATE SUBSCRIPTION app_sub CONNECTION 'host=primary
    dbname=app' PUBLICATION app_pub;
```

#033

Leader Election

WHAT IT IS

A group of nodes agrees on one "leader" to perform a task or accept writes.

WHY IT MATTERS

Avoids two nodes doing the same exclusive job (split-brain).

EXAMPLE

```
if (Cache::lock('scheduler-leader', 60)->get()) {
    // this instance is the leader for 60s; renew
    while alive
}
```

Found this useful?

Save it for later.

Share it with a teammate.

Full guide with all code examples:
the Backend Engineering Handbook (GitHub).